

Assignment 3: Human Activity Recognition

Name: [Your Name]

Student ID: [Your Student ID]

Kaggle Display Name: [Your Kaggle Name]

1. Code Repository and Instructions

Public GitHub Link: [Insert Public GitHub Link Here]

1.1 Environment Setup

To accurately reproduce our results, please ensure you have Python 3 installed along with the required libraries. The core dependencies for our pipeline are `numpy`, `pandas`, `lightgbm`, `scikit-learn`, and `scipy`.

1.2 Execution Pipeline

You must place the Kaggle dataset files, namely `train_data.npz` and `test_data.npz`, in the `nycu-data-mining-assignment-3/` directory relative to the repository root. Once the data is in place, execute the `04_baseline_model.ipynb` notebook, which encapsulates our best-performing approach.

Running this notebook sequentially will:

1. Load and normalize the raw 3-axis accelerometer and gyroscope data.
2. Automatically extract both global and temporally-localized sub-window statistical features.
3. Train a Light Gradient Boosting Machine (LightGBM) Classifier using a rigorously cross-validated 5-fold `GroupKFold` strategy.
4. Output evaluation metrics for each fold and generate the `submission_ultimate.csv` file required for the Kaggle upload.

2. Preliminary Analysis (10%)

2.1 The Challenge of Sensor Noise and Inter-Personal Variability

Before arriving at our final LightGBM approach, we conducted extensive preliminary analyses using state-of-the-art Deep Learning architectures, specifically 1D Convolutional Neural Networks (CNNs) combined with Long Short-Term Memory (LSTM) networks.

We initially hypothesized that the sequential nature of the 300-second windows would be perfectly suited for deep learning.

However, during our exploratory data analysis, we observed that the raw sensor data was extremely noisy and exhibited profound inter-personal variability. Human activity data collected from wrist-worn accelerometers is notoriously difficult to model because every individual possesses idiosyncratic movement signatures—different walking gaits, varying arm swing amplitudes, and distinct sitting postures. Consequently, our deep learning approaches proved highly prone to overfitting. The neural networks tended to memorize the precise, highly-specific movement signatures of the training users and failed catastrophically when attempting to generalize to unseen users in the validation set. This consistently resulted in sub-optimal macro F1-scores.

2.2 The Shift to Gradient Boosting

In contrast, preliminary tests utilizing tree-based gradient boosting models, specifically LightGBM, demonstrated vastly superior generalization provided they were fed robust, manually engineered statistical features. This critical observation led to our core design decision to pivot entirely away from Deep Learning. Instead of forcing a neural network to learn spatial filters from scratch on noisy data, we chose to actively engineer statistical features that capture the fundamental physical properties of the movements (such as overall energy, variance, or dominant frequency peaks). These statistical properties remain far more consistent across different individuals than raw sequential traces.

2.3 Interpretation of the 0.7088 Benchmark

Surpassing the Baseline 3 threshold (a macro F1-score of 0.7088) is a significant milestone. In a 6-class classification problem involving noisy, real-world sensor data, achieving a score well above 0.7 indicates that our model successfully isolates the underlying distributions of the distinct activities. It proves the model can effectively differentiate between highly ambiguous states (such as distinguishing sitting from standing) which share nearly identical gravitational orientations along the Z-axis. The model achieves this purely by leveraging subtle differences in movement jerkiness and subtle high-frequency dynamic shifts.

3. Preprocessing Techniques (10%)

3.1 Cross-Validation Strategy

To prevent data leakage, we partitioned the training data using `GroupKFold` based on the `user_ids`. If standard K-Fold were used, contiguous fragments of a single user's activity could end up in both the training and validation sets, artificially inflating the score and ruining generalization. `GroupKFold` ensured a highly reliable local cross-validation score that accurately mirrored our Kaggle leaderboard performance.

3.2 Mathematical Feature Engineering

Our preprocessing pipeline largely eschewed raw signal scaling in favor of extensive statistical feature engineering, which yielded the most significant improvements in model performance. For feature engineering, we extracted a comprehensive array of statistics across both time and frequency domains:

Time-Domain Statistics: We calculated fundamental properties including mean (μ), standard deviation (σ), minimum, maximum, median, quartiles, skewness, and kurtosis. We also captured the total signal energy by computing the sum of squares across the window:

$$E = \sum_{i=1}^N x_i^2 \quad (1)$$

Signal Derivatives (Jerk): To capture sudden, jerky movements which are highly indicative of transitions or dynamic activities like walking, we computed the mean absolute difference:

$$Jerk = \frac{1}{N-1} \sum_{i=1}^{N-1} |x_{i+1} - x_i| \quad (2)$$

Frequency-Domain (FFT): Human movement, especially walking, is highly rhythmic. We utilized the Fast Fourier Transform (FFT) to convert the time-domain signals into the frequency domain. Extracting the mean, standard deviation, and maximum values of the FFT magnitudes allowed our model to easily distinguish periodic movements from chaotic noise.

Sensor Correlations and Magnitude: We calculated the Pearson correlation coefficient (ρ) between the X, Y, and Z axes to capture the 3D orientation of the wrist. Furthermore, we calculated the overall Euclidean magnitude of acceleration ($M = \sqrt{x^2 + y^2 + z^2}$), which provides a rotation-invariant measure of overall movement intensity.

The inclusion of these engineered features provided a massive F1-score jump compared to naive baselines that merely relied on global averages.

4. Alignment of Activity Labels with Sequential Readings (10%)

4.1 The Alignment Problem

The dataset provides only a single activity label for every 5-minute CSV file. This means 300 seconds of high-frequency sequential readings must be mapped to one single target. Directly utilizing standard Machine Learning models (like Random Forests or SVMs) on 300 raw sequential points flattened into an array is highly ineffective, as the model loses all concept of time and sequence.

4.2 Temporal Alignment via Sub-Windowing

To properly align the sequential accelerometer readings with the single overarching label, we transformed the temporal sequence into a static feature vector utilizing an advanced

Sub-Windowing technique.

First, we computed our entire suite of statistical features over the *entire 300-second sequence*. This captures the global, overarching activity trend (e.g., mostly sedentary vs. mostly active).

Subsequently, to capture temporal dependencies without the need for a recurrent neural network, we divided the 300-second window into **five smaller, sequential blocks of 60 seconds each**. We then computed the exact same statistical features for each of these 60-second blocks independently.

By concatenating the 5 sub-window feature vectors with the global feature vector, we effectively embedded the passage of time into our tabular data. This temporal alignment strategy allowed our LightGBM model to understand the general trend of the entire 5-minute window while simultaneously recognizing localized, transient temporal changes (e.g., a user sitting for 3 minutes, then standing up and walking for 2 minutes) over the five distinct intervals.

5. Ablation Study for Core Technical Design Choices (10%)

Our experimental journey involved systematically testing several core design choices, which ultimately proved that our `04_baseline_model` was the most effective and robust solution.

5.1 Deep Learning vs. Gradient Boosting

As previously mentioned, an ablation replacing LightGBM with CNN and LSTM architectures resulted in a severe drop in the macro F1-score. We actively chose LightGBM because decision trees are inherently more robust to unscaled, high-dimensional tabular data. LightGBM can seamlessly handle the non-linear relationships and differing scales present in our engineered statistical features (e.g., comparing an FFT magnitude to a Pearson correlation) without suffering from the vanishing gradients or severe overfitting seen in our LSTM trials.

5.2 Global vs. Sub-Window Features

We performed an ablation where we relied solely on the 300-second global features. This led to a substantial decrease in performance. The global features averaged out too much critical localized information. If a brief but highly indicative movement occurred in only one minute of the window, the 5-minute average completely obscured it. This proved that the 60-second sub-windows were absolutely essential for temporal resolution.

5.3 Granularity and The Overfitting Trap

To push performance further, we hypothesized that even finer temporal resolution would be beneficial. In our subsequent `06_grandmaster_model`, we experimented with increasing the sub-window granularity to 10 blocks of 30 seconds and engineered an even larger number of features (surpassing 1000 features).

Counterintuitively, this highly granular approach actually decreased performance on the validation set. The excessive feature count combined with the highly fragmented 30-second blocks caused the LightGBM algorithm to aggressively overfit the training data. The model began finding spurious correlations in the noise of the fragmented blocks rather than learning generalized activity patterns.

5.4 Ensemble Constraints and the "Sweet Spot"

We additionally attempted an `05_ensemble_model` approach, combining predictions from LSTMs, simple Random Forests, and LightGBM. However, this ensemble performed worse than the standalone LightGBM model, as the inclusion of weaker underlying models degraded the highly accurate predictions of the boosted trees.

Ultimately, the `04_baseline_model` represented the perfect **sweet spot**. By utilizing five 60-second sub-windows, applying a `class_weight='balanced'` parameter to handle imbalanced activities, and enforcing a feature subsampling rate of 80 percent (`colsample_bytree=0.8`) to explicitly combat overfitting, the model provided enough temporal granularity to capture transient activities while remaining sufficiently constrained to generalize exceptionally well to new users.